

Informe Sweet Crush Desafío 1

Integrantes: Sergio Cabrera (1086922688) / Samuel Bastidas (1138525766)

1 Contextualización

El Desafío I exige implementar el juego Sweet Crush representando cada ficha del tablero con exactamente 3 bits, dentro de una secuencia continua de memoria dinámica, sin desperdiciar espacio en alineaciones a byte y sin desempaquetar jamás el tablero en una estructura auxiliar donde cada posición ocupe un byte o un entero completo. Esta restricción condiciona todo el diseño: obliga a resolver la manipulación de bits mediante los operadores de bits y la administración de memoria dinámica, reservando y liberando la cantidad de memoria que corresponde a las dimensiones actuales del tablero.

La solución está escrita en C++ y se desarrolló como proyecto de consola en Qt Creator: la visualización se hace por consola con `cin` y `cout`, y toda la memoria dinámica se maneja con `new[]` y `delete[]`.

2 Análisis del problema y consideraciones para la solución

2.1 Requerimientos funcionales

El programa debe crear un tablero de dimensiones indicadas por el usuario al iniciar; mostrarlo en dos formatos, el de fichas y el binario, después de cada jugada; permitir al jugador eliminar la ficha de una posición dada; detectar automáticamente combinaciones de tres o más fichas iguales en horizontal o vertical, con la posibilidad de que una misma ficha participe a la vez en una combinación horizontal y en una vertical; tras una eliminación, aplicar gravedad y rellenar los huecos superiores con fichas aleatorias de distribución uniforme; repetir el ciclo de detección, eliminación y gravedad (cascada) hasta que no aparezcan combinaciones nuevas; permitir insertar o eliminar filas y columnas en cualquier posición y mantener el estado de la partida con un criterio de puntuación.

2.2 Restricciones

Representación. Cada ficha ocupa exactamente 3 bits y el tablero es una secuencia continua de bits, sin relleno para alinear cada ficha a un byte. La trama válida empieza en el bit menos significativo del primer byte y, si el total de bits no es múltiplo de ocho, los bits sobrantes quedan agrupados a la izquierda, es decir, en las posiciones más significativas del último byte.

Memoria. La memoria reservada debe corresponder a las dimensiones vigentes: no se permite mantener dimensiones máximas, sobredimensionar el tablero ni reservar espacio para expansiones futuras. Al eliminar filas o columnas, la memoria física solo debe modificarse cuando la utilización caiga por debajo del 65 % respecto al tamaño actual. Se interpretó “tamaño actual” como la capacidad realmente reservada en el bloque en ese momento, y “utilización” como la fracción de esa capacidad que necesitan las nuevas dimensiones.

Lenguaje. C++ en el framework Qt, sin sintaxis de ANSI C; sin `struct`, `class`, `template`, objetos definidos por el estudiante, contenedores de la STL ni objetos `string`; solución multiarchivo (`.h` y `.cpp`); uso de punteros, arreglos y memoria dinámica.

2.3 Alternativa adoptada y descartada

Se adoptó una única representación física, un arreglo dinámico de `unsigned char`, sobre el que en cada acceso se calculan el byte y el desplazamiento de bit correspondientes a una posición lógica (fila, columna). Las alternativas consideradas y descartadas fueron las siguientes.

Copia desempaquetada del tablero (un char o un int por ficha). El enunciado la prohíbe de forma explícita y anularía el propósito del desafío.

Bloque de capacidad máxima fija (1000 × 1000) reutilizado. Está prohibido: la memoria debe corresponder a las dimensiones vigentes.

Leer siempre dos bytes consecutivos (por ejemplo con un unsigned short) para evitar los casos de partición. En la última ficha del tablero se leería un byte fuera del bloque, y además se ocultaría la partición entre bytes que el enunciado pide manejar (Figura 3 del enunciado).

Marcar con el código 111 directamente en el tablero durante el barrido horizontal. El barrido vertical ya no vería el valor original de esas celdas y se perderían combinaciones simultáneas. Se resolvió con un mapa auxiliar temporal de 1 bit por celda

3. Esquema de tareas y organización modular

3.1 Tareas

T1 (módulo bits). Calcular cuántos bytes necesita un tablero y en qué bit empieza una ficha: `calcularTamanoBytes` y `calcularOffsetBit`.

T2 (módulo bits). Leer y escribir una ficha de 3 bits, aunque quede repartida entre dos bytes: `leerCelda` y `guardarCelda`.

T3 (módulo tablero). Crear y liberar la memoria del tablero: `inicializarTablero` y `destruirTablero`.

T4 (módulo tablero). Generar una ficha aleatoria y validar posiciones: `generarCeldaRandom` y `esPosicionValida`.

T5 (módulo tablero). Insertar y quitar filas y columnas en cualquier posición, con la regla del 65 %: `agregarFila`, `quitarFila`, `agregarColumna` y `quitarColumna`.

T6 (módulo juego). Llenado inicial aleatorio del tablero: `llenarTableroInicial`.

T7 (módulo juego). Eliminar la ficha que indica el jugador: `quitarCeldaJugador`.

T8 (módulo juego). Detectar y limpiar combinaciones horizontales, verticales y simultáneas: `detectarYLimpiarRachas`.

T9 (módulo juego). Hacer caer las fichas y rellenar los huecos: `aplicarCaidaFichas`.

T10 (módulo juego). Repetir detección y gravedad (cascadas) y acumular estadísticas y puntaje: `ejecutarCascadas`.

T11 (módulo interfaz). Mostrar el tablero en formato de fichas y en formato binario, y mostrar las estadísticas: `simboloDeCelda`, `imprimirTabSimbolos`, `imprimirTabBinario`, `imprimirTablero` e `imprimirEstadisticas`.

T12 (módulo main). Leer entradas con validación, mostrar el menú y coordinar los demás módulos: `pedirEntero`, `pedirEnteroRango`, `imprimirMenu` y `main`.

3.2 Organización modular y dependencias

bits.h / bits.cpp contiene el cálculo del tamaño y del offset de bit, y la lectura y escritura de una ficha. No conoce el concepto de fila y columna del tablero como tal: solo recibe el total de columnas. No depende de ningún otro módulo.

tablero.h / tablero.cpp administra el ciclo de vida de la memoria: creación, liberación, inserción y eliminación de filas y columnas, validación de posiciones y generación de la ficha aleatoria. Depende de `bits`.

juego.h / juego.cpp contiene las reglas de la partida: llenado inicial, eliminación por el jugador, detección de combinaciones, gravedad y cascadas con sus estadísticas. Depende de `bits` y de `tablero`.

interfaz.h / interfaz.cpp se encarga de la presentación por consola de las dos vistas del tablero y de las estadísticas. Depende de `bits`.

main.cpp hace la lectura validada de datos, muestra el menú y coordina a los otros cuatro módulos con un switch. Depende de los cuatro.

3.3 Flujo de una jugada

bits.h / bits.cpp contiene el cálculo del tamaño y del offset de bit, y la lectura y escritura de una ficha. No conoce el concepto de fila y columna del tablero como tal: solo recibe el total de columnas. No depende de ningún otro módulo.

tablero.h / tablero.cpp administra el ciclo de vida de la memoria: creación, liberación, inserción y eliminación de filas y columnas, validación de posiciones y generación de la ficha aleatoria. Depende de `bits`.

juego.h / juego.cpp contiene las reglas de la partida: llenado inicial, eliminación por el jugador, detección de combinaciones, gravedad y cascadas con sus estadísticas. Depende de `bits` y de `tablero`.

interfaz.h / interfaz.cpp se encarga de la presentación por consola de las dos vistas del tablero y de las estadísticas. Depende de `bits`.

main.cpp hace la lectura validada de datos, muestra el menú y coordina a los otros cuatro módulos con un switch. Depende de los cuatro.

4 Algoritmos implementados

4.1 Localización y acceso a una ficha (bits.cpp)

Dada una posición (fila, col) y el total de columnas, `calcularOffsetBit` calcula el índice lineal `fila * totalColumnas + col` y lo multiplica por `BITS_POR_CELDA` (3) para obtener el bit inicial global de la ficha. A partir de él, `leerCelda` y `guardarCelda` obtienen el byte y el desplazamiento con dos operaciones de bits:

```
int Index = bitInicial >> 3;      // byte que contiene el primer bit de la ficha
int desplazamiento = bitInicial & 7; // posición del primer bit dentro de ese byte
```

Como 8 fichas ocupan exactamente 24 bits (3 bytes), la situación se repite cada 8 fichas. Dentro de cada grupo de 8, las fichas 0, 1, 3, 4, 6 y 7 (desplazamientos 0, 3, 1, 4, 2 y 5) caben completas dentro de un byte. La ficha 2 (bit inicial 6, desplazamiento 6) ocupa los bits 6 y 7 del primer byte del grupo y el bit 0 del byte siguiente. La ficha 5 (bit inicial 15, desplazamiento 7) ocupa el bit 7 del segundo byte del grupo y los bits 0 y 1 del tercero. Por eso hay tres casos:

Desplazamiento menor o igual que 5. La ficha completa cabe en un byte. Se extrae con `(tab[Index] >> desplazamiento) & MASCARA_CELDA`.

Desplazamiento igual a 6. Hay 2 bits en el byte actual y 1 en el siguiente. Se combinan con `bitsBajos | (bitAlto << 2)`.

Desplazamiento igual a 7. Hay 1 bit en el byte actual y 2 en el siguiente. Se combinan con `bitBajo | (bitsAltos << 1)`.

`guardarCelda` replica esta partición en sentido inverso: para cada byte involucrado limpia con una máscara invertida (`~(...)`) únicamente los bits que le corresponden a la ficha y luego inyecta el valor nuevo con `|`, sin tocar los bits de las fichas vecinas que comparten el byte. Antes de escribir aplica `dato &= MASCARA_CELDA`, de modo que un valor fuera de rango nunca puede invadir bits ajenos. Como el bit 0 de la numeración es el bit menos significativo del primer byte, los bits sobrantes del último byte quedan siempre en sus posiciones más significativas, como exige el enunciado.

Ninguna de las dos funciones accede fuera del bloque: solo tocan el byte siguiente (`Index + 1`) cuando la ficha está partida, y en ese caso ese byte contiene los bits restantes de la propia ficha, por lo que forma parte de los bytes que `calcularTamanoBytes` reservó.

4.2 Codificación de las fichas (bits.h, juego.h, interfaz.cpp)

Los códigos 0 a 5 (binario 000 a 101) representan las seis fichas jugables y se muestran con los símbolos X, O, #, @, & y \$, respectivamente. El código 6 (110) es `CELDA_VACIA`, se muestra con el punto (.) y solo existe mientras se resuelve una cascada. El código 7 (111) es `CELDA_MARCADA`, se muestra con el signo de interrogación (?) y es la marca temporal exclusiva del algoritmo de detección. La constante `VALOR_MAXIMO_FICHA = 5`, definida en `juego.h`, permite distinguir con una sola comparación (`valor > VALOR_MAXIMO_FICHA`) una ficha jugable de cualquiera de los dos códigos especiales. `simboloDeCelda` devuelve el signo de admiración (!) para cualquier código fuera de esta relación, como protección defensiva.

4.3 Generación de fichas aleatorias (tablero.cpp, main.cpp)

`generarCeldaRandom` devuelve `rand() % 6`, es decir, un código entre 0 y 5. `srand` se invoca **una sola vez** al inicio de `main`, con `time(nullptr)`; el resto de la partida solo llama a `rand`, de modo que las fichas sucesivas no repiten semilla.

4.4 Detección y limpieza de rachas (detectarYLimpiarRachas, juego.cpp)

El algoritmo tiene tres fases y trabaja sobre un mapa auxiliar de 1 bit por celda (`mapaMarcas`, de `(filas * columnas + 7) / 8` bytes, inicializado en 0).

Fase 1, barrido horizontal. Para cada fila se recorren las columnas con un `while` cuyo índice salta hasta el final de cada racha de valor igual. Si la longitud es mayor o igual que `RACHA_MINIMA` (3), cada celda de la

racha se marca en el mapa con `marcarEnMapa`, que ejecuta `mapa[indice >> 3] |= (1u << (indice & 7))`, donde `indice` es `fila * columnas + columna`. El tablero real no se modifica.

Fase 2, barrido vertical. Es análoga, por columnas, y marca sobre el mismo mapa. Como el tablero sigue intacto, este barrido ve los valores originales de todas las celdas, incluidas las que el barrido horizontal ya marcó.

Fase 3, materialización. Se recorre el mapa con `estaMarcadoEnMapa` y toda celda marcada se escribe en el tablero como `CELDA_MARCADA`. Se libera el mapa con `delete[]` y un último recorrido del tablero convierte todo `CELDA_MARCADA` en `CELDA_VACIA`. Aunque el mapa ya identifica las celdas por sí solo, se conserva la escritura intermedia del código 111 para mantener el papel que el diseño preliminar le asignó.

Las celdas con valor mayor que `VALOR_MAXIMO_FICHA` (vacías o marcadas) se saltan con `continue` y nunca inician una racha. Cada racha, horizontal o vertical, suma 1 a `rachasEncontradas`; una racha de seis fichas iguales cuenta como una sola. En cambio, celdas Marcadas cuenta cada celda **una sola vez**, aunque pertenezca a una racha horizontal y a una vertical, porque se cuenta al recorrer el mapa; así, una figura en L o en T se procesa completa y sus puntos no se duplican. La función devuelve `true` si marcó al menos una celda, y ese valor es la condición del ciclo de cascadas.

El costo de una ronda de detección es proporcional a filas por columnas: cada dirección visita cada celda una vez, porque los saltos hasta el final de cada racha evitan volver a comparar las celdas ya cubiertas, y a eso se suman el recorrido del mapa y el último recorrido del tablero.

El mapa no incumple la prohibición de desempaquetar el tablero: no representa fichas, ocupa 1 bit por celda se construye y consulta únicamente con operadores de bits, y se reserva y libera dentro de cada llamada, por lo que no es una representación permanente del tablero.

4.5 Gravedad (`aplicarCaidaFichas`, `juego.cpp`)

Para cada columna se recorre desde la última fila hacia arriba. Cada posición vacía busca hacia arriba la ficha no vacía más cercana: si la encuentra, la traslada a la posición vacía y deja vacía la de origen; si no encuentra ninguna, todo lo que queda por encima está vacío y la posición se rellena con una ficha nueva de `generarCeldaRandom()`. El resultado observable es que las fichas caen verticalmente conservando su orden y los huecos superiores de la columna se rellenan con fichas nuevas. El preliminar describía este mismo recorrido con el matiz de que el relleno se haría como un paso posterior; en el código se hace dentro de la misma pasada con el mismo resultado.

Sobre su costo: cada posición vacía puede recorrer hacia arriba la porción de huecos consecutivos que tenga encima, de modo que el costo crece con la longitud de esas rachas de huecos (en el peor caso, cuadrático en la altura de la columna) y es prácticamente lineal cuando hay pocos huecos por columna, que es lo habitual: una eliminación manual deja un hueco y una racha deja tres o unos pocos más. La sección 10 comenta una posible mejora.

4.6 Ciclo de cascadas (`ejecutarCascadas`, `juego.cpp`)

Mientras `detectarYLimpiarRachas` siga encontrando combinaciones, `ejecutarCascadas` acumula las celdas marcadas, las rachas encontradas y el puntaje, y llama a `aplicarCaidaFichas`. En cada iteración se incrementan `cascadasEnJugada` (que se reinicia a 0 al comenzar cada llamada a `ejecutarCascadas`) y `totalCascadas` (acumulado de la partida). Se usa un `while` porque el número de rondas no se conoce de antemano.

Criterio de conteo. Se cuenta como cascada toda ronda de detección en la que se encuentra al menos una combinación, incluida la primera tras una jugada, porque esa primera combinación ya es consecuencia de la reorganización del tablero que sigue a la eliminación (definición de cascada del enunciado). Por eso una eliminación que produzca tres cascadas sucesivas se reconoce en pantalla como “Cascadas (jugada actual): 3”.

`ejecutarCascadas` se invoca después de cualquier operación que modifique el tablero (eliminar una ficha, insertar o quitar una fila o una columna) y una vez al inicio del programa, para resolver las combinaciones que hayan salido por puro azar en el llenado inicial. Las estadísticas de esas rondas iniciales (fichas eliminadas, combinaciones, cascadas y puntos) quedan acumuladas, aunque el jugador todavía no haya jugado.

4.7 Redimensionamiento del tablero: lógica de índices (`tablero.cpp`)

Inserción (agregarFila y agregarColumna). Valida que la posición esté entre 0 y el total actual, ambos inclusive, y que no se supere TAM_MAXIMO (1000). Las filas o columnas anteriores a la posición conservan su índice y las posteriores lo aumentan en uno. Las fichas existentes se copian con leerCelda y guardarCelda, y la fila o columna nueva se llena con fichas aleatorias. Insertar una fila intermedia desplaza en bloque las filas posteriores; insertar una columna reconstruye el tablero fila por fila, porque el ancho de cada fila cambia y casi todas las fichas cambian de posición física.

Eliminación (quitarFila y quitarColumna). Valida la posición y que el tablero no quede por debajo de TAM_MINIMO (1). Se recorren las fichas omitiendo la fila o columna indicada y escribiendo cada ficha conservada en su nuevo índice lógico. Que esa escritura se haga sobre un bloque nuevo o sobre el mismo bloque lo decide la regla del 65 %.

4.8 Interfaz (interfaz.cpp)

imprimirTabSimbolos calcula dos anchos por separado: anchoFila = contarCifras(filas - 1) para la columna de índices de fila, y anchoCol = contarCifras(columnas - 1) + 1, con un mínimo de 3, para el encabezado y cada símbolo. Así los índices no se superponen en tableros con 100 o más columnas. imprimirTabBinario imprime cada byte físico del arreglo, del bit 7 al bit 0, en líneas de 8 bytes; por eso, en pantalla, la primera ficha del tablero corresponde a los tres bits del **extremo derecho** del primer byte, y los bits sobrantes aparecen a la **izquierda** del último byte. El encabezado indica los bytes necesarios para las dimensiones vigentes. imprimirTablero invoca ambas vistas, lo que cumple el requisito de mostrar el tablero en los dos formatos después de cada jugada. Todas las funciones de la interfaz reciben el tablero como const unsigned char*, por lo que no pueden modificarlo.

4.9 Estado del juego y criterio de puntuación

main mantiene nueve variables de tipo int: ocho de estado de la partida y una de memoria. Con ellas se cubre lo mínimo que exige el enunciado (dimensiones, eliminaciones del usuario, fichas eliminadas, combinaciones, cascadas de la jugada y puntuación) más totalCascadas, y se pasan por referencia a las funciones que las actualizan.

numFilas y numCols guardan las dimensiones actuales del tablero y las actualizan inicializarTablero y las funciones agregar y quitar. **capacidadBuffer** guarda los bytes realmente reservados en el bloque; no es una variable de juego, pero es indispensable para la regla del 65 % y la actualizan inicializarTablero, las funciones agregar y las funciones quitar cuando reducen. **jugadasUsuario** cuenta las eliminaciones manuales del jugador y la actualiza quitarCeldaJugador. **totalCeldasEliminadas** cuenta todas las fichas eliminadas, la manual más las de cada combinación, y la actualizan quitarCeldaJugador y ejecutarCascadas. **totalRachasDetectadas** cuenta las combinaciones (rachas horizontales o verticales) detectadas y la actualiza ejecutarCascadas. **cascadasEnJugada** cuenta las rondas de detección y caída de la última operación y ejecutarCascadas la reinicia en cada llamada. **totalCascadas** acumula las rondas de toda la partida. **puntajeAcumulado** guarda la puntuación y la actualiza ejecutarCascadas.

Criterio de puntuación. PUNTOS_POR_CELDA = 10 puntos por cada ficha eliminada dentro de una combinación (constante definida en juego.h). La ficha que el jugador elimina manualmente se cuenta en "Fichas eliminadas (total)", pero no otorga puntos por sí misma, porque los puntos se calculan al procesar las combinaciones (celdasMarcadas * PUNTOS_POR_CELDA).

4.10 Entrada de datos y menú (main.cpp)

pedirEntero repite la lectura mientras cin falle (limpia el estado con cin.clear() y descarta la línea con cin.ignore) y también descarta cualquier resto en la misma línea; pedirEnteroRango además valida el rango. Las dimensiones iniciales se validan contra TAM_MINIMO y TAM_MAXIMO (1 a 1000) y las posiciones contra las dimensiones vigentes. El menú es un switch de ocho opciones (de la 0 a la 7, donde 0 es salir) dentro de un ciclo controlado por la bandera enPartida; al salir se llama a destruirTablero.

5 Gestión de la memoria dinámica y optimización

5.1 Un único bloque contiguo de tamaño mínimo

Todo el tablero vive en un solo bloque contiguo de memoria dinámica, apuntado por un unsigned char* (matrizJuego, en main). Cada byte del bloque es solo almacenamiento físico: no representa una ficha, sino

un fragmento de la secuencia continua de bits en la que las fichas están empaquetadas de a 3 bits, una a continuación de la otra.

El tamaño del bloque lo calcula `calcularTamanoBytes` con aritmética entera: multiplica filas, columnas y 3 para obtener los bits totales y redondea hacia arriba a bytes sumando 7 antes de dividir entre 8. Es la cantidad mínima posible de bytes para esos bits, de modo que el desperdicio total del tablero es de máximo 7 bits, los que sobran en el último byte, sin importar el tamaño del tablero. Por ejemplo, un tablero de 1×1 necesita 3 bits y ocupa 1 byte; uno de 5×5 necesita 75 bits y ocupa 10 bytes; uno de 10×10 necesita 300 bits y ocupa 38 bytes; y el máximo permitido, de 1000×1000 , necesita 3 000 000 de bits y ocupa 375 000 bytes. Con un `char` o `int` por ficha ocuparía mucha más

5.2 Propiedad del bloque RAM y ciclo de vida

El puntero al bloque pertenece a `main`, que además guarda las dimensiones (`numFilas` y `numCols`) y la capacidad real en bytes (`capacidadBuffer`). No hay variables globales: todo el estado se pasa a las funciones como parámetros. La forma de pasar el puntero indica qué puede hacer cada función con él. Las funciones que pueden reasignarlo (`inicializarTablero`, `destruirTablero`, `agregarFila`, `quitarFila`, `agregarColumna` y `quitarColumna`) lo reciben por referencia, como `unsigned char*&`. Las que solo modifican el contenido (`guardarCelda`, `llenarTableroInicial`, `quitarCeldaJugador`, `detectarYLimpiarRachas`, `aplicarCaídaFichas` y `ejecutarCascadas`) lo reciben como `unsigned char*`. Las que solo leen (`leerCelda` y todas las de la interfaz) lo reciben como `const unsigned char*`, de modo que el compilador impide que modifiquen el tablero. En todos los casos se pasa un puntero y nunca

Creación. `inicializarTablero` rechaza la llamada si el puntero ya apunta a un bloque, para no perder la referencia a memoria reservada y producir una fuga; valida que las dimensiones estén entre 1 y 1000; calcula los bytes necesarios; reserva el bloque con `new unsigned char[bytes]`; lo inicializa en 0 y actualiza las dimensiones y la capacidad. Inicializar en 0 hace que los bits sobrantes del último byte partan en 0.

Uso. Toda lectura y escritura pasa por `leerCelda` y `guardarCelda` con posiciones válidas, verificadas por `esPosicionValida` o por los rangos de las funciones de entrada, de modo que no se accede fuera del bloque.

Redimensionamiento. Se describe en las secciones 5.4 y 5.5

Destrucción. `destruirTablero` ejecuta `delete[]` solo si el puntero no es nulo, lo deja en `nullptr` y pone dimensiones y capacidad en 0, de modo que no queda un puntero colgante ni es posible liberar el mismo bloque dos veces. `main` la invoca al salir del ciclo del menú, que solo termina con la opción 0. Si la creación inicial falla, el programa termina sin haber reservado nada.

5.3 Puntos de reserva y liberación

Solo hay cinco puntos del programa que usan `new[]` o `delete[]`, y cada reserva tiene su liberación en el mismo flujo de ejecución. `inicializarTablero` reserva el bloque del tablero y `destruirTablero` lo libera. `agregarFila` y `agregarColumna` reservan `nuevoTablero` y liberan el bloque anterior una vez copiado su contenido. `quitarFila` y `quitarColumna` reservan `recompactado` únicamente cuando toca reducir, y en ese caso liberan el bloque anterior tras la copia; cuando no toca reducir, no reservan ni liberan nada. `detectarYLimpiarRachas` reserva `mapaMarcas` y lo libera antes de retornar, dentro de la misma llamada. Ninguna otra función del programa pide memoria dinámica.

5.4 Crecimiento: inserción de filas y columnas

Un bloque creado con `new[]` no se puede ampliar en sitio. Por eso, al insertar una fila o una columna se sigue siempre el patrón reservar, copiar, liberar y reasignar. Se reserva un bloque del tamaño exacto para las nuevas dimensiones (`calcularTamanoBytes` con las dimensiones nuevas) y se inicializa en 0. Se copian las fichas existentes, una a una, con `leerCelda` y `guardarCelda` hacia su nueva posición lógica, y se llena la fila o columna nueva con fichas aleatorias. Se libera el bloque anterior con `delete[]` y el puntero del que llama se reasigna al bloque nuevo, que es la razón por la que se recibe por referencia. Por último se actualizan las dimensiones y `capacidadBytes`.

La copia se hace ficha a ficha, y no copiando bytes, porque al insertar una fila o una columna intermedia las fichas posteriores cambian de posición de bit y su alineación respecto de los límites de byte cambia; una copia de bytes solo serviría cuando las fichas conservan su posición de bit, que no es el caso general. El costo es proporcional al número de fichas y es inevitable con este empaquetado.

Consecuencia para la memoria. Como el bloque nuevo se reserva con el tamaño exacto para las nuevas dimensiones, toda inserción devuelve la capacidad a exactamente lo necesario, aun cuando el bloque anterior tuviera holgura por eliminaciones previas. Su costo transitorio es que mientras dura la copia coexisten el bloque viejo y el nuevo.

5.5 Reducción: eliminación de filas y columnas y regla del 65 %

Al eliminar una fila o una columna primero se valida la posición y que el tablero no quede con menos de una fila o una columna. Luego, antes de tocar el bloque, se decide si corresponde reducir la memoria física. Se calcula `bytesUsadosNuevos`, que son los bytes que necesitan las nuevas dimensiones, y se evalúa la regla del 65 % con aritmética entera, sin números decimales ni redondeos:

```
bool reducirMemoria = (bytesUsadosNuevos * 100 < capacidadBytes * 65);
```

Aquí `capacidadBytes` es el tamaño realmente reservado del bloque en ese momento, que puede ser mayor que lo estrictamente necesario si hubo eliminaciones previas que no redujeron.

Si toca reducir, porque el uso cae por debajo del 65 % de la capacidad, se reserva un bloque nuevo de exactamente `bytesUsadosNuevos`, se copian las fichas conservadas omitiendo la fila o la columna eliminada, se libera el bloque anterior y `capacidadBytes` toma el nuevo valor.

Si no toca reducir, se recompacta directamente sobre el mismo bloque, sin ningún `new[]` ni `delete[]`. Cada ficha conservada se lee y se escribe en su nuevo índice lógico, los bytes que quedan al final del bloque (desde `bytesUsadosNuevos` hasta `capacidadBytes - 1`) se ponen en 0 y `capacidadBytes` no cambia. La memoria física sigue siendo la misma, con una parte sin usar, que es lo que el enunciado permite mientras la utilización no caiga por debajo del 65 %.

Por qué recompactar en el mismo bloque es seguro. Las fichas conservadas se procesan en orden creciente de su posición lineal original, y el índice donde se escribe cada una es siempre menor o igual que el índice de donde se leyó. Al quitar una fila, la ficha (`fi`, `ci`) pasa a (`fd`, `ci`) con `fd` menor o igual que `fi`, y su índice lineal `fd * columnas + ci` no supera a `fi * columnas + ci`. Al quitar una columna, la ficha pasa a una columna `cd` menor o igual que `ci` en filas más estrechas, con lo que `fi * (columnas - 1) + cd` tampoco supera a `fi * columnas + ci`. Por eso cada escritura cae sobre una posición que ya se leyó, o sobre la misma, y nunca sobre una ficha pendiente de leer. Además, `guardarCelda` solo modifica los 3 bits de la ficha destino y deja intactos los de sus vecinas dentro del mismo byte.

Invariante de utilización. Con esta regla, después de cualquier operación de redimensionamiento los bytes necesarios son al menos el 65 % de la capacidad reservada. Tras crear el tablero o insertar, `capacidad` y `bytes` necesarios coinciden. Tras eliminar, o bien se reduce y vuelven a coincidir, o bien no se reduce precisamente porque el uso sigue siendo al menos el 65 %. Por tanto, la memoria sin usar nunca supera el 35 % del bloque.

Por qué se mide contra la capacidad reservada. Si la regla comparara solo contra el tamaño inmediatamente anterior de las dimensiones, cada eliminación aislada reduciría el uso en pocos puntos porcentuales y nunca dispararía la reducción; tras muchas eliminaciones seguidas el bloque podría quedar con una fracción mínima de uso sin que la regla se activara jamás. Al llevar `capacidadBytes` como variable de estado y compararla en cada eliminación, la holgura acumulada queda acotada.

Por qué no reducir en cada eliminación. Reducir implica reservar un bloque, copiar todo el tablero y liberar el anterior. En un tablero de 10×10 , quitar una fila ahorra 4 bytes (de 38 a 34); pagar una reserva, una copia completa y una liberación por un ahorro tan pequeño no compensa. La regla del 65 % evita ese costo.

5.6 El mapa auxiliar de la detección de combinaciones

Además de las funciones de creación y redimensionamiento del tablero, `detectarYLimpiarRachas` es la única función que reserva memoria. Su mapa `Marcas` ocupa 1 bit por celda, es decir $(\text{filas} * \text{columnas} + 7) / 8$.

El mapa se reserva al comenzar cada llamada y se libera dentro de la misma llamada, en cuanto termina la fase de materialización y antes del último recorrido del tablero, de modo que su tiempo de vida es el mínimo necesario.

Se eligió reservarlo por llamada, y no conservarlo de forma permanente, por dos razones. Primera, un mapa permanente tendría que redimensionarse después de cada inserción o eliminación de filas o columnas, y

mientras tanto ocuparía memoria que no corresponde a las dimensiones vigentes. Segunda, la reserva y la liberación cuestan poco frente al recorrido completo del tablero que la propia ronda de detección ya hace. El costo es un `new[]` y un `delete[]` por ronda, incluida la última ronda, la que no encuentra combinaciones.

5.7 Pila y ausencia de reservas

Todo lo demás vive en la pila: contadores, índices, valores temporales de 3 bits y punteros. No hay arreglos de tamaño fijo, variables globales, objetos string ni contenedores de la STL. La interfaz imprime el tablero directamente con `cout`, ficha a ficha en el formato de símbolos y bit a bit en el formato binario. En consecuencia, en reposo el único bloque de memoria dinámica del programa es el del tablero, y solo durante una operación existe.

5.8 Resumen de las optimizaciones y equilibrio con la eficiencia en instrucciones

Optimizaciones de memoria. El empaquetado en 3 bits por ficha y el bloque del tamaño mínimo posible dejan como único desperdicio los bits sobrantes del último byte. La reserva es siempre exacta al crear el tablero y al insertar, y la holgura posterior a las eliminaciones queda acotada al 35 % por la regla del 65 %, medida contra la capacidad realmente reservada. Cuando no toca reducir, la eliminación no reserva ni libera memoria. El mapa de la detección ocupa un bit por celda y se libera en la misma llamada. Los tipos se eligieron por tamaño y el tablero nunca se copia al pasarlo a una función.

Optimizaciones de instrucciones. El acceso a una ficha es directo y de costo constante: un cálculo de offset, un desplazamiento y una máscara, sin recorridos. El byte y el desplazamiento se obtienen con `>> 3` y `& 7` en lugar de una división y un residuo. `leerCelda` y `guardarCelda` tocan como máximo dos bytes y nunca leen más allá del bloque. Los barridos de detección saltan hasta el final de cada racha, por lo que visitan cada celda una vez por dirección. La regla del 65 % se evalúa con enteros. Y la recompactación en el mismo bloque evita la reserva, la liberación y una segunda copia completa del tablero cuando no toca reducir.

Decisiones en las que se equilibraron memoria e instrucciones. El empaquetado de 3 bits obliga a calcular el offset y a enmascarar en cada acceso, su costo se mantiene pequeño con las operaciones de bits. Las inserciones y eliminaciones copian ficha a ficha, con costo proporcional al número de fichas, porque el desplazamiento de las fichas no conserva su alineación. El mapa de detección se reserva por llamada, con un costo pequeño frente al recorrido del tablero, a cambio de no mantener memoria fuera de las dimensiones vigentes. No reducir la memoria en cada eliminación favorece las instrucciones, con la holgura acotada por el 65 %; reducirla siempre favorecería la memoria a un costo de copia mucho mayor.

5.9 Limitaciones conocidas de la gestión de memoria

Coexistencia transitoria. Durante una inserción coexisten el bloque anterior y el nuevo y durante la detección coexisten el tablero y el mapa. Ambos casos son transitorios.

Bits sobrantes tras una recompactación en sitio. Cuando `quitarFila` o `quitarColumna` recompactan en el mismo bloque, se ponen en 0 los bytes posteriores al último usado, pero los bits sobrantes del último byte usado conservan el valor que tenían antes. Siguen ubicados en las posiciones más significativas, como exige el enunciado, y el programa nunca los lee, porque todo acceso pasa por `leerCelda` con índices válidos; solo pueden verse distintos de cero en la vista binaria. En los bloques recién reservados (creación, inserción y reducción) esos bits parten en 0.

6 Selección y justificación de tipos de datos y estructuras de control

6.1 Tipos de datos

Bloque del tablero: unsigned char*. El byte es la unidad mínima direccionable, por lo que el desperdicio es de menos de 8 bits en todo el tablero. Al no tener signo, un desplazamiento a la derecha nunca arrastra un bit de signo. Se descartó un `char` por ficha, un `int` por ficha y un bloque de `unsigned int` debido a que aumentan el costo en la memoria.

Valor de una ficha, máscaras y bytes temporales: unsigned char. Tres bits caben en el tipo más pequeño disponible; un `int` usaría cuatro veces más memoria en cada variable temporal, sin ventaja. Como los resultados de `~` y `<<` se promueven a `int`, las máscaras invertidas se convierten explícitamente con `static_cast` a `unsigned char` antes de usarse.

Dimensiones, índices, offsets de bit, contadores y puntaje: int. El mayor valor de bits es $1000 \times 1000 \times 3 = 3\,000\,000$, que cabe de sobra en un int, y filas * columnas * 3 no desborda. Se descartó short (máximo 32 767), que se desbordaría en el cálculo de bits, y long long, que duplicaría memoria sin necesidad.

Retornos bool. Expresan éxito o fracaso (quitarCeldaJugador, inserciones y eliminaciones) y sirven como condición directa del ciclo de cascadas (detectarYLimpiarRachas). Se descartaron los códigos numéricos de error por ser menos legibles y costosos.

Constantes const int y const unsigned char. Tienen tipo y alcance, el compilador las verifica y dejan un solo lugar para cambiar cada valor (BITS_POR_CELDA, MASCARA_CELDA, CELDA_VACIA, CELDA_MARCADA, RACHA_MINIMA, PUNTOS_POR_CELDA, VALOR_MAXIMO_FICHA, TAM_MINIMO y TAM_MAXIMO).

Parámetros por referencia (int& y unsigned char*&). Permiten devolver varios valores y reasignar el puntero del que llama tras un new[] y un delete[], sin variables globales. Se descartaron las variables globales porque acoplan los módulos y dificultan verificar quién modifica qué.

Mensajes: literales const char*. Solo se usan literales de cadena en cout y como parámetro de pedirEntero; no existe ningún objeto string.

6.2 Estructuras de control

for. Se usa en los recorridos con límites conocidos: filas, columnas, bytes del bloque y copia al recompartar. Deja el rango a la vista en una sola línea.

while. Se usa en los barridos de rachas (donde el índice salta hasta el final de la racha), en la búsqueda hacia arriba de la gravedad, en el ciclo de cascadas y en el ciclo del menú: en todos, el número de iteraciones no se conoce de antemano y un for obligaría a modificar el contador dentro del cuerpo.

if, else if y else. Se usan en leerCelda y guardarCelda, que tienen exactamente tres casos excluyentes (desplazamiento menor o igual que 5, igual a 6 e igual a 7); un switch no expresa bien el rango “menor o igual que 5”.

switch. Se usa en el menú principal y en simboloDeCelda, donde los valores son discretos y enumerables; una cadena de if sería más larga y menos clara.

continue, break y la bandera enPartida. continue salta las celdas especiales en la detección; break cierra cada caso del switch; la bandera termina el ciclo del menú porque un break dentro del switch solo saldría del switch, no del ciclo.

Funciones static internas. pedirEntero, pedirEnteroRango, imprimirMenu, marcarEnMapa, estaMarcadoEnMapa, contarCifras y otras son auxiliares de un único archivo. Al declararlas static no se exponen en los encabezados y se evitan colisiones de nombres.

7 Problemas de desarrollo que se afrontaron

El informe preliminar anticipaba tres dificultades: la precisión de las máscaras al escribir una ficha repartida entre dos bytes, el control de índices al recompartar el tablero en posiciones intermedias y la sincronización de punteros en las reasignaciones de memoria. La primera y la tercera se confirmaron como se anticiparon; la segunda se manifestó de forma más específica (problema 4). Además aparecieron dos problemas que el preliminar no había previsto (los problemas 1 y 5).

Problema 1. Interferencia entre el barrido horizontal y el vertical al marcar rachas. El primer diseño escribía el código 111 directamente en el tablero apenas se detectaba una racha horizontal. El barrido vertical posterior ya no encontraba el valor original de esas celdas (leerCelda devolvía CELDA_MARCADA, mayor que VALOR_MAXIMO_FICHA), por lo que las rachas verticales que incluían una celda ya marcada se detectaban incompletas o se perdían. La solución fue el mapa auxiliar de 1 bit por celda que separa la fase de detectar de la de escribir.

Problema 2. Máscaras de escritura entre bytes. Una máscara invertida mal construida en los casos de desplazamiento 6 y 7 corrompe en silencio bits de la ficha vecina, sin error de compilación ni de ejecución.

Solución: se verificaron los tres casos de partición con tableros pequeños, inspeccionando a mano el volcado binario.

Problema 3. Sincronización de punteros en las reasignaciones. Cada `new[]` debe corresponder a un `delete[]` del bloque anterior; un desfase produce fugas o punteros colgantes. **Solución:** se aplicó el mismo patrón reservar, copiar, liberar y reasignar en las operaciones de redimensionamiento, y el puntero se recibe por referencia para que la reasignación llegue a `main`

Problema 4. Recompactar en el mismo bloque sin buffer temporal. Al evitar el buffer temporal cuando no se reduce memoria, había que garantizar que escribir una posición no destruyera datos aún no leídos. La solución fue se comprobó que el índice de escritura es siempre menor o igual que el de lectura, tanto dentro de una fila (columnas) como entre filas distintas, por lo que la operación es segura

8 Evolución de la solución y discrepancias con el informe preliminar

8.1 Discrepancias de diseño o implementación

8.1.1 Marcado de combinaciones: mapa auxiliar de 1 bit por celda

Qué decía el preliminar. La sección 3.4 proponía un algoritmo de dos pasadas. En la primera, toda ficha que formara parte de una racha se reemplazaba temporalmente por el código 111 dentro del propio tablero, en el momento de detectarla. En la segunda, todo 111 se reescribía como 110.

Qué hace el código. Los barridos horizontal y vertical no modifican el tablero: marcan los índices de las celdas en el mapa `mapaMarcas`, de un bit por celda. Solo cuando ambos barridos terminan se escribe `CELDA_MARCADA` (111) en el tablero real, y un último recorrido la convierte en `CELDA_VACIA` (110). El código 111 conserva el papel que el preliminar le asignó, marca temporal exclusiva de la eliminación; lo que cambió es el momento y el lugar donde se guarda la marca durante la detección.

Por qué se cambió. Si se escribe 111 en el tablero durante el barrido horizontal, el barrido vertical ya no puede comparar esas celdas con sus vecinas, porque dejan de tener el valor de ficha original y pasan a valer 111. Una racha vertical que incluya una celda ya marcada por el barrido horizontal se detectaría incompleta o no se detectaría. Eso incumple el requisito del enunciado de que una misma ficha pueda participar a la vez en una combinación horizontal y en una vertical y sea procesada correctamente. El propio preliminar planteaba ese requisito en la sección 2.1, y decía que exigía marcar las coincidencias antes de eliminarlas físicamente; el mapa cumple ese mismo objetivo sin la interferencia entre direcciones.

Costo y por qué es aceptable. El mapa consume memoria adicional transitoria y una reserva y liberación por ronda de detección. No viola la prohibición de desempaquetar el tablero: no representa fichas, ocupa 1 bit por celda, se construye y consulta solo con operadores de bits y no persiste entre llamadas.

8.1.2 Operadores de bits explícitos en lugar de división y residuo

Qué decía el preliminar. En la sección 3.2, el byte que contiene una ficha se obtiene dividiendo el bit inicial entre ocho, y el desplazamiento interno, con el residuo de esa división.

Qué hace el código. `leerCelda`, `guardarCelda`, `marcarEnMapa` y `estaMarcadoEnMapa` usan `>> 3` para el byte y `& 7` para el desplazamiento.

Por qué se cambió. Para enteros no negativos y un divisor que es potencia de dos, dividir entre 8 y desplazar 3 bits a la derecha dan el mismo resultado, y lo mismo ocurre con el residuo y la máscara `& 7`; el comportamiento no cambia. El criterio de operadores a nivel de bits (30 %) pide máscaras y desplazamientos, y expresar el cálculo así hace explícita la intención de manipular bits sin costo extra y evita depender de que el compilador convierta la división en un desplazamiento en la operación más repetida del programa.

8.1.3 Recompactación en el mismo bloque cuando no se reduce la memoria

Qué decía el preliminar. La eliminación sigue un procedimiento equivalente en sentido inverso al de la inserción, omitiendo durante la copia la fila o columna indicada y recompactando el resto, y la reducción física del bloque solo se ejecuta cuando la memoria utilizada cae por debajo del 65 % de la capacidad reservada. Leído junto con la inserción, el procedimiento equivalente implica copiar hacia un bloque nuevo; el preliminar no decía qué se hace cuando no toca reducir.

Qué hace el código. quitarFila y quitarColumna deciden antes de copiar si toca reducir. Solo en ese caso reservan un bloque nuevo. Cuando no toca reducir, recom pactan sobre el mismo bloque, ponen en 0 los bytes sobrantes al final y no ejecutan ningún new[] ni delete[]

Por qué se cambió. Eliminar filas o columnas es una operación frecuente, y reservar y liberar memoria solo para conservar el mismo tamaño de bloque es un costo innecesario. Una versión intermedia del desarrollo reservaba siempre un bloque temporal y, si no tocaba reducir, lo copiaba de vuelta al original para descartarlo enseguida; eso costaba una reserva, una liberación y una segunda copia completa del tablero sin cambiar el tamaño del bloque.. Se la puso como diferencia porque el preliminar, no la contemplaba; no contradice la regla del 65 %, la respeta con menos costo.

8.1.4 Relleno de huecos dentro de la misma pasada de la gravedad

Qué decía el preliminar. En la sección 3.4, la gravedad recorre cada columna de abajo hacia arriba; ante una posición vacía busca la ficha válida más próxima por encima, la baja y deja vacía la posición de origen, “para luego rellenar los huecos que permanezcan en la parte superior de la columna” con fichas nuevas. Es decir, el relleno se describe como un paso posterior al desplazamiento.

Qué hace el código. El desplazamiento de las fichas es el mismo, pero el relleno se hace dentro de la misma pasada: cuando una posición vacía no encuentra ninguna ficha por encima, se rellena de inmediato con una ficha nueva, y lo mismo ocurre con las posiciones vacías que quedan sobre ella al continuar el recorrido.

Por qué se cambió. El resultado es equivalente: las fichas conservadas terminan en las mismas posiciones y las fichas nuevas siguen la misma distribución uniforme. Además se evita un segundo recorrido de la parte alta de cada columna solo para localizar huecos que el primer recorrido ya conoce. No afecta a ningún criterio de evaluación.

8.2 Aclaraciones y adiciones

8.2.1 Cascadas tras el llenado inicial

El preliminar decía que la incorporación de fichas nuevas mediante relleno aleatorio debía disparar el ciclo de detección, pero no decía qué hacer con las combinaciones que puedan salir por azar en el llenado inicial. El código llama a ejecutarCascadas una vez después de llenarTableroInicial y antes de mostrar el tablero, para que la primera jugada parta de un tablero estable. La consecuencia es que las estadísticas de esas rondas (fichas eliminadas, combinaciones, cascadas y puntos) quedan acumuladas aunque el jugador todavía no haya jugado, por lo que pueden ser distintas de cero antes de la primera eliminación.

8.2.2 Criterio de conteo de cascadas

El preliminar decía que cada repetición del ciclo de detección se registraría como una cascada adicional, sin decir si la primera ronda de una jugada cuenta. El código cuenta como cascada toda ronda de detección en la que se encuentra al menos una combinación, incluida la primera, porque esa combinación ya es consecuencia de la reorganización del tablero tras la eliminación, que es como el enunciado define la cascada. Por eso una demostración de tres cascadas sucesivas equivale a ver “Cascadas (jugada actual): 3” en las estadísticas.

8.2.3 Variables de estado adicionales (adición)

El preliminar hablaba de seis variables de estado. El código conserva todas y agrega totalCascadas, que acumula las rondas de toda la partida; cascadasEnJugada sigue existiendo y cumple el mínimo del enunciado. También agrega capacidadBuffer, que no es una variable de juego sino de memoria y es indispensable para evaluar la regla del 65 % contra la capacidad realmente reservada. Ninguna sustituye a una variable exigida.

8.2.4 Lectura validada de datos y opciones adicionales del menú (adición)

El preliminar decía que la entrada y la salida se harían con cin y cout y que el menú permitiría al jugador hacer jugadas y solicitar modificaciones estructurales. El código agrega lectura validada: pedirEntero y pedirEnteroRango limpian el estado de cin, descartan lo que sobre en la línea y validan el rango. Sin ello, una entrada no numérica deja cin en estado de error y el menú entra en un ciclo infinito, y una posición fuera de rango podría leer o escribir fuera del bloque, lo que afecta directamente a la gestión de la memoria. Además, el menú incluye las opciones 6 (mostrar el tablero), 7 (mostrar las estadísticas) y 0 (salir), que no modifican el

estado de la partida. Por esta validación, main contiene funciones de lectura que el preliminar no mencionaba, pero sigue sin implementar lógica nueva de juego.

9 Cumplimiento de los criterios de evaluación.

9.1 Uso de operadores a nivel de bits (30 %)

Operador >>. Obtiene el byte (bitInicial >> 3), alinea una ficha al bit 0 antes de enmascararla, separa los bits altos de la ficha en los casos de desplazamiento 6 y 7, y calcula el índice de byte en el mapa auxiliar. Está en leerCelda, guardarCelda, marcarEnMapa y estaMarcadoEnMapa.

Operador <<. Desplaza la máscara y el valor a su posición, reubica fragmentos de ficha y construye la máscara de un bit del mapa y de la impresión binaria. Está en guardarCelda, leerCelda, marcarEnMapa, estaMarcadoEnMapa e imprimirTabBinario.

Operador &. Obtiene el desplazamiento (& 7), aísla 3 bits con MASCARA_CELDA, aísla fragmentos de ficha y consulta un bit del mapa o de un byte. Está en leerCelda, guardarCelda, estaMarcadoEnMapa e imprimirTabBinario.

Operador |. Une los fragmentos de una ficha partida entre bytes, inyecta el valor nuevo y activa un bit del mapa. Está en leerCelda, guardarCelda y marcarEnMapa.

Operador ~. Construye la máscara invertida que limpia los 3 bits de destino sin tocar los vecinos, en los tres casos de partición. Está en guardarCelda.

Operador ^. No se usa

9.2 Gestión eficiente de la memoria (35 %)

En resumen dónde se cumple cada punto del criterio. La memoria dinámica se usa con new[] y delete[], con cada reserva emparejada con su liberación. El bloque tiene el tamaño mínimo en bytes y la reserva es exacta al crear el tablero y al insertar. Al eliminar, la reducción física ocurre solo bajo el 65 % de la capacidad reservada, con aritmética entera, y la holgura queda acotada al 35 %. Cuando no toca reducir, la recompactación se hace in situ sin reservar ni liberar memoria. El mapa auxiliar ocupa un bit por celda y se libera en la misma llamada. Los tipos se eligieron por tamaño y por seguridad frente a desbordes.

9.3 Lógica de juego (35 %)

Crear dinámicamente un tablero: inicializarTablero (tablero.cpp).

Generar las fichas iniciales de forma aleatoria con distribución uniforme: generarCeldaRandom (tablero.cpp), usada por llenarTableroInicial (juego.cpp).

Mostrar el tablero: imprimirTablero (interfaz.cpp).

Eliminar la ficha indicada por el usuario: quitarCeldaJugador (juego.cpp), tras validar la posición con esPosicionValida.

Detectar combinaciones horizontales y verticales: el primer y el segundo barrido de detectarYLimpiarRachas.

Resolver combinaciones simultáneas: el mapa auxiliar de detectarYLimpiarRachas y el ciclo de ejecutarCascadas.

Reorganizar las fichas restantes: aplicarCaidaFichas.

Agregar y eliminar una fila, agregar y eliminar una columna: agregarFila, quitarFila, agregarColumna y quitarColumna (tablero.cpp), en cualquier posición válida, incluidas las intermedias.

Gestión dinámica de la memoria del tablero: el patrón reservar, copiar y liberar de las cuatro operaciones de redimensionamiento, la regla del 65 % y destruirTablero.

Visualización en formato binario y en formato fichas, ambos después de cada jugada: main llama a imprimirTablero tras cada operación de las opciones 1 a 5, una vez resueltas las cascadas.

Cascadas: ejecutarCascadas, con los contadores cascadasEnJugada y totalCascadas.

Estado del juego y puntuación: las variables de la sección 4.9 y 10 puntos por ficha eliminada dentro de una combinación.

10 Consideraciones

Gestión eficiente de la memoria: Se logró guardar cada ficha en exactamente 3 bits dentro de un solo bloque de memoria dinámica, desperdiciando como máximo unos pocos bits al final del arreglo. Además, la regla del 65 % se calculó solo con números enteros para ajustar el tamaño de la memoria sin hacer reservas o liberaciones innecesarias si se podía reutilizar el mismo espacio.

Acceso y modificación por bits: El uso directo de los operadores de bits (>>, <<, &, |, ~) permitió leer y cambiar cualquier ficha al instante, resolviendo de forma segura los casos donde una ficha queda dividida entre dos bytes sin dañar la información de las fichas de al lado.

Detección precisa de combinaciones: Se utilizó un mapa temporal de 1 bit por casilla para revisar el tablero sin alterar las fichas durante el proceso. Esto evitó que la revisión horizontal interfiriera con la vertical, permitiendo detectar combinaciones en forma de L o T sin romper la regla de desempaquetar el tablero.

Organización modular y código seguro: El programa se dividió claramente en cinco partes independientes (bits, tablero, juego, interfaz y main). Esto, junto con la validación de los datos que ingresa el usuario y el buen uso de punteros, evitó errores de acceso fuera del tablero y fugas de memoria durante la partida.